



TD

4



4



6

sp

كلية الهندسة المعلوماتية

السنة الأولى

ال class في جافا

د. ماهر صارم



البرمجة 2

RB Informatcs; 20/4/2026

محتوى مجاني غير مخصص للبيع التجاري

## الصف (Class) في جافا

تعريف class:

هو القالب البرمجي الذي يحدد الخصائص (متغيرات) والسلوكيات (توابع) التي ستمتلكها الكائنات (objects) التي سيتم إنشاؤها بناءً عليه.

الclass في جافا مشابهة جداً لـ Struct في c++

ولكن الclass في جافا مخصص لبناء كائنات (Objects) مخفية أما في C++ الStruct هو حاوية بيانات ظاهرة.

هناك ثلاث طرق للتحكم في كيفية الوصول للمتغيرات والدوال في جافا:

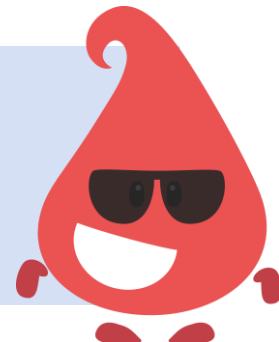
1. عام (Public):

أي class (صف) آخر في المشروع يمكنه الوصول إلى المتغير أو التابع المعرف عن طريق Public.

2. خاص (Private):

لا يمكن الوصول إليه إلا من داخل الكلاس نفسه، أي لا يمكن لكلاسات أخرى الوصول إليه إلا عن طريق تابع نقوم بصنعه يعرف باسم getter أو تابع setter سنتعرف عليهم لاحقاً في المحاضرة.

Behind every smile...  
there're teeth.



## 3. ثابت (Static):

هذه الطريقة لا تشبه الـ public والـ private بل هي تعمل عملاً آخر، فهي لا تتعلق بالقدرة على رؤية المتغير بل تتعلق بإمكان وجود البيانات.

**أي:** تعني أن هذا المتغير أو الدالة تخص الكلاس نفسه وليس الكائنات (objects) التي ننشئها فيه وهذا يعني أنك لا تحتاج لـ object حتى تستخدمه، وإذا قمت بتغيير قيمته في مكان ما ستتغير قيمته لكل الكائنات (objects) ويتم تعريفه مرة واحدة في الكلاس على عكس المتغيرات العادية التي تعرف كل مرة يتم فيها صنع كائن من هذا الكلاس.

## مثال توضيحي لـ class يعبر عن مستطيل:

## ملاحظات:

- يفضل دوماً أن يكون اسم الكلاس مبدوءاً بحرف كبير.
- عند إنشاء class جديد في جافا يجب التأكد من أنه في نفس الـ package الخاصة بكلاس main.
- لكل كلاس يوجد باني (instructor)، ودوماً ما يكون بنفس اسم الكلاس.
- في حالة عدم إنشاء instructor يقوم الكلاس بإنشاء باني افتراضي له.
- في المثال السابق نلاحظ وجود بانيتين اثنتين، الأولى (السطر 4) لا يحوي أي وسائط فيه ويتم وضع قيم افتراضية للمتغيرات ضمنه، أما الثاني في السطر 9 يمرر عبره وسيطين يعبران عن الطول والعرض.

```

1- public class Rect {
2-     double width;
3-     double length;
4-     Rect(){
5-         width=10;
6-         length=20;
7-         System.out.println(width*length);
8-     }
9-     Rect(double width,double length)
10-    {
11-        this.width=width;
12-        this.length=length;
13-    }
14- }

```

(داخل الـ main)

```

1- public class Main{
2-     public static void main
3-     (String[]args)
4-     {
5-         Rect rect1= new Rect();
6-         Rect rect2= new Rect(3.2,8.9);
7-         System.out.println(rect2.width);
8-         rect2.length=19.8;
9-     }
10- }

```

ماذا تعني `this.width`؟

نلاحظ أنه ضمن الباني عرفنا متغير بنفس اسم المتغيرات ضمن الكلاس، لذا عندما نريد إسناد قيمة لمتغيرات الكلاس نضع قبلها **this** للدلالة على المتغير الخاص بالكائن المنشأ.

سطر 3 ضمن `main` يعبر عن طريقة صنع كائن جديد ونلاحظ استعمالنا للباني الافتراضي.

سطر 4 استعمالنا الباني الذي يحوي وسائل بسبب وضع قيم ضمن أقواس الباني.

سطر 5+6 الطرق للوصول للمتغيرات أو التوابع في الكلاس هي بكتابة اسم الكائن متبوع باسم التابع أو المتغير ويفصل بينهما نقطة (.)

من المثال السابق:

شرح الكود:

قمنا بجعل المتغير `width` معرفاً خاص (private)، أي أنه ظاهر فقط ضمن الكلاس الخاص فيه ولا يمكن الوصول إليه من class آخر.

في السطر 9 عرفنا متغير مسؤول عن إعادة قيمة `width` لأنه في حال قمنا بكتابة `x.width` في `main` لن يعمل البرنامج لأنه لن يكون قادراً على التعرف على `width` (بسبب اعتباره متغير خاص) لذا قمنا بصنع تابع يعيد قيمة `width`.

نلاحظ عند استدعاء التابع في السطر 7 ضمن `main` سيعمل البرنامج لأن التابع `getWidth` هو تابع `public`.

في السطر 12 عرفنا متغير مسؤول عن تعديل قيمة `width` واستدعيناه ضمن `main` في السطر 5. (هنا شرحنا `sitter` والـ `getter`)

```
1- public class Rect {
2-     private double width;
3-     double length;
4-     Rect(double width,double length)
5-     {
6-         this.width=width;
7-         this.length=length;
8-     }
9-     public double getWidth(){
10-        return this.width;
11-    }
12-     public double sitWidth(double
width){
13-         this.width=width;
14-     }
}
```

(داخل الـ `main`)

```
1- public class Main{
2-     public static void main(String[]args){
3-         Rect x = new Rect(3.2,1.2);
4-         x.length=8;
5-         x.setWidth(5.2);
6-         System.out.println(x.length);
7-         System.out.println(x.getWidth());
8-     }
}
```

## مثال توضيحي لفكرة الـ Static:

## شرح الكود:

```

1- public class Student {
2-     int id;
3-     String name;
4-     static int num=1;
5-     student(){
6-     student(String name)
7-     {
8-         id=num++;
9-         this.name=name;
10-    }
11-    public void printInfo(){
12- System.out.println("id="+id);
13- System.out.println("name="+name);
14-    }
15- }

```

(داخل الـ main)

```

1- public class Main{
2- public static void
main(String[] args){
3- Student S1 = new
Student("Ahmad");
4- Student S2 = new Student("Ali");
5- Student S3 = new Student("Majd");
6- S1.printInfo();
7- S3.printInfo();
8- System.out.println(S1.num);
9- System.out.println(S2.num);
10- System.out.println(Student.num);
11- }
12- }

```

كلاس Student يعبر عن الطلاب، يحوي متغيرات تعبر عن الاسم والرقم التسلسلي وعدد الطلاب.

عرفنا عدد الطلاب عن طريق static كي يتعرف مرة واحدة ضمن الكلاس مع المحافظة على قيمته عند صنع كائن جديد.

في السطر 6 عرفنا بانني يحوي متغير للاسم وجعلنا الـ id يساوي عدد الطلاب الحالي وبعدها أضفنا لعدد الطلاب 1 (num++).

ملاحظة: id=num++; أولاً يسند القيمة بعدها يزيد القيمة لـ num.

عند صنع كائن S1 كما في السطر 3 ضمن main فإنه يتم إسناد قيمة 1 لـ id الخاص بـ S2 وبعدها زيادة قيمة الـ num لتصبح مساوية لـ 3.

سطر 11 ضمن كلاس Student عرفنا متغير يقوم بطباعة معلومات الطالب.

سطر 7 ضمن الـ main لطباعة معلومات الطالب 3.

سطر 9+10+8 ضمن الـ main كلها ستطبع نفس الرقم وهو 4 لأن المتغير num لا يتعلق بكائن معين، بل هو يتعلق بالكلاس ككل.

انتهت المحاضرة الرابعة، بالتوفيق! ☺